

Software Engineering: SDLC

1. Lifecycle activities

Typical activities include requirements analysis, design, implementation, testing, deployment and maintenance. These activities may repeat or overlap. Waterfall emphasizes sequential phases; iterative development revisits and improves the product through repeated cycles. Agile favors incremental delivery and feedback, not the absence of planning or documentation.

2. Requirements

Functional requirements describe behavior: a student can upload a PDF note. Non-functional requirements describe qualities or constraints: the upload accepts at most 20 MB, or responses meet a defined performance target. Requirements should be testable and should include relevant conditions.

3. Verification and validation

Verification checks whether work products satisfy their specifications: are we building the product right? Validation checks whether the product meets user needs: are we building the right product? Reviews and tests can support these activities; verification is not simply a synonym for testing.

4. Testing levels

Unit tests check individual components. Integration tests check interactions between components. System tests check the complete integrated product. Acceptance tests assess readiness against user or business acceptance criteria. These levels describe scope, not necessarily a single rigid execution order.

5. Masterji integration example

A unit test checks the greeting classifier. An integration test checks the chat API with a mocked provider. An end-to-end test opens a PDF, asks a question and verifies the visible response. Mocked tests alone cannot prove the real provider and deployed app work together.

Testing: Techniques & Examples

1. Black-box and white-box

Black-box tests derive cases from externally visible behavior without depending on implementation details. White-box tests use internal structure, such as branches or paths. High code coverage is useful evidence but does not guarantee correct requirements or defect-free software.

2. Equivalence partitioning

Suppose a form accepts integer marks from 0 through 100 inclusive. Partitions include valid integers 0-100, integers below 0, integers above 100, and invalid types such as non-numeric text. Choose representative cases from each relevant partition.

3. Boundary value analysis

For the marks field, test -1, 0, 1, 99, 100 and 101. These probe the lower and upper edges and neighboring values. Also check empty input and non-integers if the specification forbids them. Passing 50 alone does not test boundary handling.

4. Regression and smoke tests

Regression testing checks that changes have not broken previously working behavior. Smoke testing checks critical paths to decide whether a build is suitable for further testing. A smoke suite is not a substitute for comprehensive regression coverage.

5. Defect reporting

A useful report includes environment, prerequisites, reproducible steps, expected result, actual result and evidence. Severity measures impact; priority concerns when to address the defect. A severe issue and an urgent issue need not be the same.

Exam practice

Design boundary tests for an allowed file size of 1-20 MB inclusive. Distinguish system testing from acceptance testing. Explain why 100 percent statement coverage does not prove every branch or requirement has been tested.